

Loyola University Chicago
Department of Computer Science

COMP 170: Introduction to Object-Oriented Programming
Practice Questions

October 1, 2026



What these questions are for. These practice questions walk you through the concepts we have covered in class so far, one topic at a time, so you can see what you should be reviewing and which ideas still need more work. They are grouped in six sections:

1. Errors, data types, operators, naming and variables (including `print`, strings and `input`).
2. Pseudocode and functions (parameters, `return`, annotations, docstrings, `main`).
3. `for` loops (`range`, nested loops, cumulative algorithms).
4. Conditional execution (`if/elif/else`, Boolean expressions).
5. Lists (creating, accessing, modifying, slicing and resizing lists).
6. Modules, the `math` module, Vim and terminal commands.

If a question feels hard, go back to the notes and code for that topic before moving on.

Question formats. The quiz will use the same question formats you see here: multiple-choice questions (sometimes “check all that apply”), short questions (matching, writing a value, a one- or two-sentence explanation), and code questions where you read, correct or complete code.

How to use them. Answer every question on paper first, without running anything. Then look at the answered version, type the code into a file, and run it with `python3` to test it.

Unless stated otherwise, assume all code is Python 3 and runs top to bottom. Each question is independent. Short tracing snippets may omit pseudocode, docstrings, type annotations or `main()` for brevity. Complete programs should follow the course requirements. The argument type in the `shout` question is intentionally unspecified because the function checks it at runtime.

Section 1 — Errors, data types, operators, naming & variables

1. **Types of errors.** Each version below should read a price and a quantity and print the total cost. The user types 2.5, then 4, so the correct output is Total: 10.0. Link each version to its result by writing the result number (1, 2 or 3) next to it. Each result is used exactly once.

Version A	Version B	Version C
<pre>price = float(input("Price? ")) qty = int(input("Qty? ")) print("Total: " + price * qty)</pre>	<pre>price = float(input("Price? ")) qty = int(input("Qty? ")) print("Total:", price + qty)</pre>	<pre>price = float(input("Price? ")) qty = int(input("Qty? ")) print("Total:" price * qty)</pre>

Results:

1. Python reports a `SyntaxError`, and nothing runs.
2. The program stops while running with a `TypeError` (runtime error).
3. The program runs, but prints a wrong answer (logic error).

Version A → _____ Version B → _____ Version C → _____

2. What does `print(type(7 / 7), type(7 // 7), type("7"))` display?

- ☐ `<class 'int'> <class 'int'> <class 'int'>`
☐ `<class 'float'> <class 'int'> <class 'str'>`
☐ `<class 'int'> <class 'float'> <class 'str'>`
☐ `<class 'float'> <class 'float'> <class 'int'>`

3. **Precedence.** What does `print(2 + 3 * 4 ** 2 // 8)` display?

- ☐ 8
☐ 10
☐ 20
☐ 6

4. **Operators and types.** After this code runs, what are the values of `x`, `y` and `z`? Write each value exactly as Python stores it (use quotes for strings).

```
1 x = "ab" * 3
2 y = "5" + "5" * 2
3 z = 17 % 5 * 2
```

`x` = _____ `y` = _____ `z` = _____

5. **Naming.** Which of these are **not** valid Python variable names? Check **all** that apply.

- ☐ `2nd_place`
- ☐ `first-name`
- ☐ `_count`
- ☐ `class`
- ☐ `total2`

6. **Variables.** What is printed?

```
1 a = 3
2 b = a
3 a = a * 2
4 b = b + a
5 print(a, b)
```

- ☐ 6 6
- ☐ 6 12
- ☐ 6 9
- ☐ 3 9

7. **Strings and escape sequences.** Which statement prints exactly `It's a "test"`?

- ☐ `print("It's a "test")`
- ☐ `print('It's a "test"')`
- ☐ `print("It's a \"test\"")`
- ☐ `print('It's a \"test\"')`

8. The user types `3.9` at the prompt. What happens?

```
1 n = int(input("n? "))
2 print(n + 1)
```

- ☐ It prints 4
- ☐ It prints 4.9
- ☐ It prints 5
- ☐ The program stops with a `ValueError`

9. What does `print(7 / 2 * 2, 7 // 2 * 2)` display?

- ☐ `7 7`
- ☐ `7.0 6`
- ☐ `7.0 7.0`
- ☐ `6 6`

10. Which statements print exactly `Score: 10`? Check **all** that apply.

- ☐ `print("Score: " + 10)`
- ☐ `print("Score:", 10)`
- ☐ `print("Score: " + str(10))`
- ☐ `print("Score:" + "10")`

Section 2 — Pseudocode & functions

11. What is printed?

```
1 def triple(n: int) -> int:
2     return n * 3
3     print("done")
4
5 print(triple(2))
```

- ☐ 6
- ☐ done then 6
- ☐ 6 then done
- ☐ None

12. **Local variables.** What is printed?

```
1 def set_total() -> None:
2     total = 10
3
4 total = 5
5 set_total()
6 print(total)
```

- ☐ 5
- ☐ 10
- ☐ 15
- ☐ A NameError

13. What is printed?

```
1 def main() -> None:
2     print("B")
3
4 print("A")
5 main()
6 print("C")
```

- ☐ B A C
- ☐ A B C
- ☐ A C B
- ☐ A C

14. Which statement(s) about this function are **false**? Check **all** that apply.

```
1 def average(a: float, b: float) -> float:
2     """Return the average of a and b."""
3     return (a + b) / 2
```

- ☐ average(4, 6) returns 5.0.
- ☐ The annotation `-> float` makes Python convert the result to a float.
- ☐ Calling `average(4)` raises an error, because one argument is missing.
- ☐ The docstring is printed every time `average` is called.
- ☐ `a` and `b` are parameters.

15. A function receives a person's name (text) and age (a whole number) and gives back a greeting sentence. Which header is correct?

- ☐ `def describe(name: str, age: int) -> None:`
- ☐ `def describe(name: str, age: int) -> str:`
- ☐ `def describe(name: string, age: integer) -> str:`
- ☐ `def describe(name = str, age = int) -> str:`

16. **Fill in the blanks.** Complete the program so that it asks for a temperature in Celsius and prints it in Fahrenheit ($F = C \times 9/5 + 32$).

```
1 def celsius_to_fahrenheit(c: float) -> float:
2     """Return the temperature c converted to Fahrenheit."""
3     -----(1)-----
4
5 def main() -> None:
6     """Ask for a Celsius temperature and print it in Fahrenheit."""
7     c = float(input("Celsius? "))
8     f = -----(2)-----
9     print(c, "C is", f, "F")
10
11 main()
```

17. **Pseudocode, docstrings and annotations.** The program below works, but it has no pseudocode, no docstrings and no type annotations.

```
1 def carpet_cost(length, width, price):
2     area = length * width
3     return area * price
4
5
6 def main():
7     length = float(input("Length (ft)? "))
8     width = float(input("Width (ft)? "))
9     cost = carpet_cost(length, width, 4.5)
10    print("Carpet cost: $" + str(round(cost, 2)))
11
12 main()
```

(a) Write the pseudocode for the whole program (both functions).

(b) Rewrite both function headers with type annotations, and add a docstring to each function.

18. **Correct the code.** This program should print `Perimeter: 14` for a 3 by 4 rectangle. It contains **three** bugs. Identify each bug and write the corrected program.

```
1 def rectangle_perimeter(width: float, height: float) -> float:
2     """Return the perimeter of a rectangle."""
3     perimeter = 2 * width + height
4     print(perimeter)
5
6 def main() -> None:
7     """Print the perimeter of a 3 by 4 rectangle."""
8     p = rectangle_perimeter(3, 4)
9     print("Perimeter:", p)
```

19. **Default parameter values.** What is printed?

```
1 def price_with_tax(price: float, rate: float = 0.08) -> float:
2     """Return price plus tax at the given rate, rounded to cents."""
3     return round(price * (1 + rate), 2)
4
5 print(price_with_tax(100), price_with_tax(100, 0.1))
```

- ☐ 108.0 108.0
- ☐ 108.0 110.0
- ☐ An error, because the first call is missing an argument
- ☐ 100 110.0

Section 3 — for loops

20. What is printed?

```
1 for i in range(2, 11, 3):  
2     print(i, end=" ")
```

- ☐ 2 5 8 11
- ☐ 2 5 8
- ☐ 3 6 9
- ☐ 2 4 6 8 10

21. Which loop prints 5 4 3 2 1?

- ☐ for i in range(5, 0, -1): print(i, end=" ")
- ☐ for i in range(5, 1, -1): print(i, end=" ")
- ☐ for i in range(5, 0): print(i, end=" ")
- ☐ for i in range(1, 6, -1): print(i, end=" ")

22. **Cumulative algorithms.** What is printed?

```
1 total = 1  
2 for k in range(1, 5):  
3     total *= k  
4     print(total)
```

- ☐ 24
- ☐ 10
- ☐ 120
- ☐ 0

23. After this code runs, what are the values of `total` and `i`?

```
1 total = 0
2 for i in range(10, 0, -2):
3     total += i
```

`total` = _____ `i` = _____

24. What does this code print?

```
1 for _ in range(3):
2     print("hi", end="-")
```

- ☐ hi hi hi
- ☐ hi-hi-hi-
- ☐ hi-hi-hi
- ☐ hi on three separate lines

25. **Nested loops.** Consider:

```
1 for row in range(3):
2     for col in range(4):
3         print("#", end=" ")
4     print()
```

(a) What does it print?

- ☐ 3 lines of 4 #
- ☐ 4 lines of 3 #
- ☐ 12 # on one line
- ☐ 3 lines of 3 #

(b) Rewrite the inner loop header so that the program prints this triangle instead:

```
#
##
###
```

`for col in _____:`

26. **Fill in the blanks.** Complete the loop so that, with `SIZE = 4`, it prints this right-aligned triangle. Use string multiplication.

```
*
**
***
****
```

```
1 SIZE = 4
2 for i in _____(1)_____:
3     print(" " * _____(2)_____ + "*" * i)
```

27. **Correct the code.** `sum_of_squares(3)` should return $1 + 4 + 9 = 14$. The code contains **three** bugs. Identify each bug and write the corrected function.

```
1 def sum_of_squares(n: int) -> int:
2     """Return 1*1 + 2*2 + ... + n*n."""
3     for i in range(n):
4         total = 0
5         total = i * i
6     return total
```

28. **Constants, modify-and-assign and string multiplication.** After this code runs, what are the values of `x` and `line`?

```
1 SIZE = 5
2 x = 10
3 x -= 3
4 x *= 2
5 x //= 4
6 x %= 3
7 line = "+" + "-" * SIZE + "+"
```

`x` = _____ `line` = _____

Section 4 — Conditional execution

29. What is printed?

```
1 age = 20
2 has_id = False
3 if age >= 18 or has_id and age < 18:
4     print("Enter")
5 else:
6     print("Stop")
```

- ☐ Enter
- ☐ Stop
- ☐ Enter then Stop
- ☐ An error is raised

30. With $x = 4$ and $y = 9$, which expressions are **True**? Check **all** that apply.

- ☐ $x < y$ and $y < 10$
- ☐ `not x == 4`
- ☐ $x > 5$ or $y \% 3 == 0$
- ☐ $x != 4$ or $y > 10$

31. What is printed?

```
1 score = 95
2 if score >= 90:
3     print("A")
4 if score >= 80:
5     print("B")
6 else:
7     print("C")
```

- ☐ A
- ☐ A then B
- ☐ A then C
- ☐ B

32. **Choose the structure.** For each scenario, write the letter of the structure that fits best. Not every structure is used.

A. if alone **B.** if / else **C.** if / elif / else **D.** if / elif (no else) **E.** separate if statements (if / if / if)

1. Print "Even" or "Odd" for a whole number. _____
2. Classify a yearly income as low, middle or high. _____
3. Print a warning only when the battery is below 10%. _____
4. Print "Gold", "Silver" or "Bronze" for places 1, 2 and 3; print nothing for any other place. _____
5. For a password, print "Too short", "No digit" and/or "No capital": every problem that applies. _____

33. What value does `category` hold when `temp = 72`? And when `temp = 50`? Write each value as Python stores it.

```
1 if temp < 50:
2     category = "cold"
3 elif temp < 70:
4     category = "mild"
5 elif temp < 85:
6     category = "warm"
7 else:
8     category = "hot"
```

`temp = 72:` _____ `temp = 50:` _____

34. Which condition is `True` exactly when `x` is between 1 and 10, both included?

- ☐ `x >= 1 and x <= 10`
- ☐ `x >= 1 or x <= 10`
- ☐ `x >= 1 and <= 10`
- ☐ `not (x < 1 and x > 10)`

35. **Fill in the blank.** Complete the condition using `type()` so that the function returns `word` followed by `!!!` when `word` is a string, and returns the empty string `""` otherwise.

```
1 def shout(word) -> str:
2     """Return word followed by '!!!' if it is a str; otherwise return ''."""
3     result = ""
4     if -----:
5         result = word + "!!!"
6     return result
```

Expected: `shout("hey") → "hey!!!"`, `shout(5) → ""`.

36. **Correct the code.** `ticket_price` should return 0 for ages under 5, 8 for ages 5 to 12, 12 for ages 13 to 64, and 6 for ages 65 and over. The code contains **two** bugs. Identify each bug and write the corrected function.

```
1 def ticket_price(age: int) -> int:
2     """Return the ticket price in dollars for a visitor of the given age."""
3     if age < 5:
4         price = 0
5     if age <= 12:
6         price = 8
7     elif age > 65:
8         price = 6
9     else:
10        price = 12
11    return price
```

37. **Bounding with min and max.** Which single statement keeps `score` between 0 and 100 (negative scores become 0, scores above 100 become 100, others stay the same)?

- ☐ `score = min(0, max(100, score))`
- ☐ `score = max(0, min(100, score))`
- ☐ `score = max(0, score) + min(100, score)`
- ☐ `score = min(0, score)`

Section 5 — Lists

38. What is printed?

```
1 letters = ["a", "b", "c", "d", "e"]
2 print(letters[-3:], letters[1])
```

- ☐ ['c', 'd', 'e'] b
- ☐ ['c', 'd'] a
- ☐ ['a', 'b', 'c'] b
- ☐ ['d', 'e'] b

39. What is printed?

```
1 names = ["Ana", "Ben"]
2 names.insert(1, "Cy")
3 names.append("Dee")
4 names.remove("Ana")
5 last = names.pop()
6 print(names, last)
```

- ☐ ['Cy', 'Ben'] Dee
- ☐ ['Ben', 'Cy'] Dee
- ☐ ['Cy', 'Ben', 'Dee'] Dee
- ☐ ['Ana', 'Cy'] Ben

40. After this code runs, what are the values of a, b and c?

```
1 a = [0] * 3
2 b = [1, 2] + [3, 4]
3 c = len(b)
```

a = _____ b = _____ c = _____

41. What is printed?

```
1 x = [1, 2, 3]
2 y = x
3 y[0] = 99
4 print(x)
```

- ☐ [1, 2, 3]
- ☐ [99, 2, 3]
- ☐ [99]
- ☐ An error is raised

42. What happens when this runs?

```
1 nums = [3, 6, 9]
2 print(nums[3])
```

- ☐ It prints 9
- ☐ It prints 3
- ☐ The program stops with an `IndexError`
- ☐ It prints `None`

43. **Fill in the blanks.** Complete the function so that it asks for `n` scores and returns them in a list.

```
1 def read_scores(n: int) -> list[float]:
2     """Ask for n scores and return them in a list."""
3     scores = _____(1)_____
4     for i in range(n):
5         _____(2)_____
6     return scores
```

44. **Correct the code.** `count_above([3, 7, 5, 9], 5)` should return 2. The code contains **three** bugs. Identify each bug and write the corrected function.

```
1 def count_above(values: list[int], limit: int) -> int:
2     """Return how many numbers in values are greater than limit."""
3     for i in range(len(values) - 1):
4         count = 0
5         if values[i] >= limit:
6             count += 1
7     return count
```

Section 6 — Modules, Vim & Python commands

45. Which program prints 3.0?

- ☐ `import math then print(sqrt(9))`
- ☐ `from math import sqrt then print(math.sqrt(9))`
- ☐ `import math as m then print(m.sqrt(9))`
- ☐ `import math.sqrt then print(sqrt(9))`

46. Assuming `import math`, write the value of each expression.

`math.ceil(4.2)` = _____ `math.floor(-2.5)` = _____ `round(3.5)` = _____
`math.sqrt(49)` = _____

47. Which expression is True? (Assume `import math`.)

- ☐ `0.1 + 0.2 == 0.3`
- ☐ `math.isclose(0.1 + 0.2, 0.3)`
- ☐ `round(0.1 + 0.2) == 0.3`
- ☐ `int(0.3) == 0.3`

48. A file `tools.py` contains only the line `print(__name__)`.

(a) What is printed when you run `python3 tools.py`?

- ☐ `tools`
- ☐ `__main__`
- ☐ `tools.py`
- ☐ Nothing

(b) What is printed when another file, `app.py`, contains only `import tools` and you run `python3 app.py`?

- ☐ `tools`
- ☐ `__main__`
- ☐ `app`
- ☐ Nothing, because importing a file does not run it

49. A student saves their own file as `math.py` in their project folder. In another file in the same folder they write `import math` and `print(math.sqrt(16))`, and get `AttributeError: module 'math' has no attribute 'sqrt'`. Explain in one or two sentences why this happens and how to fix it.

50. **Correct the code.** Both files are in the same folder. Running `python3 main.py` should print only the area of a circle of radius 2 (12.566...). The code in `circles.py` contains **two** problems. Identify each and write the corrected `circles.py`.

```
1 # circles.py
2 import math
3
4 def circle_area(r: float) -> float:
5     """Return the area of a circle of radius r."""
6     return pi * r ** 2
7
8 print(circle_area(1))    # quick test
```

```
1 # main.py
2 from circles import circle_area
3 print(circle_area(2))
```

51. **Vim and terminal commands.** Link each command to what it does by writing the letter next to it. Each letter is used exactly once.

<code>pwd</code> _____	A. Leave Vim without saving
<code>ls</code> _____	B. Enter Insert mode in Vim
<code>cd comp170</code> _____	C. Show which folder you are in
<code>i</code> _____	D. Save the file and leave Vim
<code>Esc</code> _____	E. List the files in the current folder
<code>:wq</code> _____	F. Move into the folder <code>comp170</code>
<code>:q!</code> _____	G. Go back to Normal mode in Vim

52. You are in your home folder. Which sequence of terminal commands creates a folder named `comp170`, moves into it, and then shows where you are?

- ☐ `cd comp170, mkdir comp170, pwd`
- ☐ `mkdir comp170, cd comp170, pwd`
- ☐ `mkdir comp170, ls comp170, pwd`
- ☐ `vim comp170, cd comp170, ls`

53. **Importing your own code.** `helpers.py` defines a working function named `average` with the signature `def average(a: float, b: float) -> float:`. It is in the same folder as `main.py`, which starts with `import helpers`. Which line in `main.py` correctly calls the function?
- ☐ `print(average(8, 9))`
 - ☐ `print(helpers.average(8, 9))`
 - ☐ `print(helpers.py.average(8, 9))`
 - ☐ `print(import helpers.average(8, 9))`
54. Which of these file names can be imported as a module with `import name` (without `.py`)? Check **all** that apply.
- ☐ `my_helpers.py`
 - ☐ `my-helpers.py`
 - ☐ `my helpers.py`
 - ☐ `helpers2.py`

This is a blank sheet for your use. Label which problem you are using it for.